

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	R S Kirithic	
Register Number	24011101082	

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

These figures are the standard published parameter counts for the original architectures on ImageNet. The versions actually trained in this experiment are adapted for CIFAR-10's much smaller 32×32 images, so their parameter counts are different – these are reported separately in Section 18.2.

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

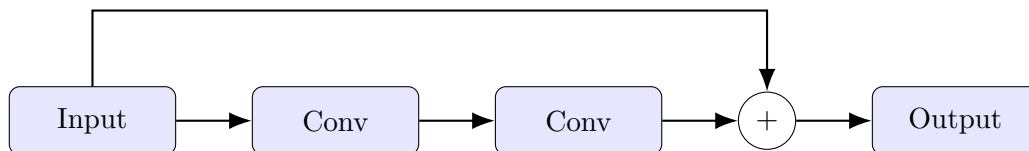


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

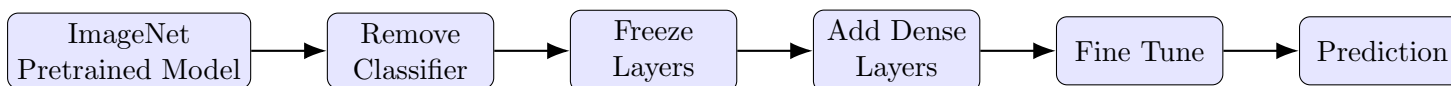


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

The dataset was loaded directly from the pickled batch files (`data_batch_1` to `data_batch_5` for training and `test_batch` for testing), stored in Google Drive, rather than downloaded again. Each batch file unpickles to a dictionary whose `data` field is a 10000×3072 array of pixel values and whose `labels` field is a list of 10000 class indices, matching the format described on the official CIFAR page. This was reshaped into $10000 \times 32 \times 32 \times 3$ images before use.

15. Experimental Procedure and Results

Task 1: Dataset Preparation

I loaded CIFAR-10 from the pickled batch files in Drive and normalised the pixel values from the original $[0, 255]$ range to $[0, 1]$. After normalisation, the pixel value range was confirmed to be $[0.00, 1.00]$. The training set had 50,000 images and the test set had 10,000 images, each $32 \times 32 \times 3$, spread across the 10 classes listed above. One sample image from each class is shown in Figure 1.

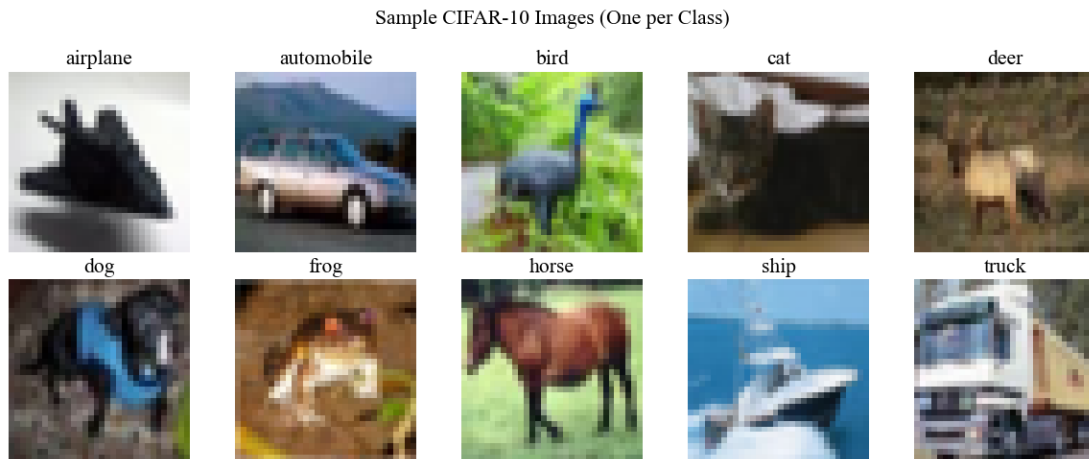


Figure 3: Sample CIFAR-10 Images (One per Class)

Observation: At 32×32 pixels the images are quite low resolution, and classes that share a similar overall shape or background, such as cat and dog, or automobile and truck, are already hard to tell apart just by looking at them.

Task 2: Transfer Learning Setup

The manual allows any one of VGG16, ResNet50, MobileNetV2 or EfficientNetB0 for this task. I used **MobileNetV2** as the main model for Tasks 2 to 5, since VGG16 and ResNet50 are already used separately in the Additional Exercises (Section 20). The pretrained ImageNet weights were loaded, the original classification layer was removed, and the convolutional base was frozen. On top of the base I

added a Global Average Pooling layer, a Dense layer of 128 units with ReLU activation, and a final Dense layer of 10 units with Softmax activation.

Since CIFAR-10 images are only 32×32 and a pretrained backbone needs a larger input to extract useful features from, images were resized to 96×96 before being passed into the network.

The resulting model had 2,423,242 parameters in total, of which 165,258 belonged to the new head and were trainable, and 2,257,984 belonged to the frozen MobileNetV2 base.

Task 3: Model Training

The training set was split into 45,000 images for training and 5,000 for validation. The model was compiled with the Adam optimizer at a learning rate of 0.001, batch size 32, categorical cross-entropy loss and accuracy as the evaluation metric, and trained for 12 epochs with the backbone still frozen, on the Google Colab GPU runtime. This stage took 250.33 seconds. By the last epoch, training accuracy had reached 92.64% and validation accuracy had reached 81.24%. Evaluated on the test set at this point (before any fine-tuning), the model reached **80.54%** accuracy.

Task 4: Fine Tuning

I unfroze the last 30 of the base model’s 154 layers, which corresponds to the last convolutional block, recompiled the model with a much lower learning rate of 1×10^{-5} so the pretrained weights would not be disturbed too aggressively, and trained for 6 more epochs. Fine-tuning took 158.69 seconds, bringing the total training time for Tasks 3 and 4 together to 409.02 seconds. By the last fine-tuning epoch, training accuracy was 86.69% and validation accuracy was 81.58%. On the test set, the fine-tuned model reached **80.26%** accuracy, compared with 80.54% before fine-tuning – a decrease of about 0.28 percentage points.

Training Curves

Figures 2 to 5 show training and validation accuracy and loss across all 18 epochs (12 frozen-training epochs followed by 6 fine-tuning epochs).

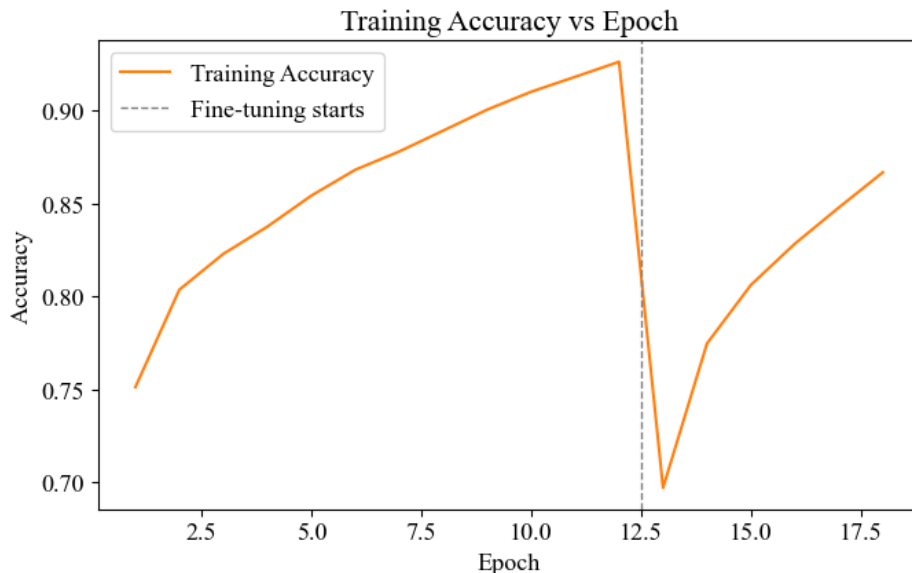


Figure 4: Training Accuracy vs Epoch

Observation: Training accuracy rose steadily from 75.12% to 92.64% while the backbone was frozen. At the point fine-tuning began (unfreezing the last block and dropping the learning rate), training accuracy fell sharply to around 69.7% before climbing back to 86.69% by the final epoch, without fully returning to the level it had reached before fine-tuning.

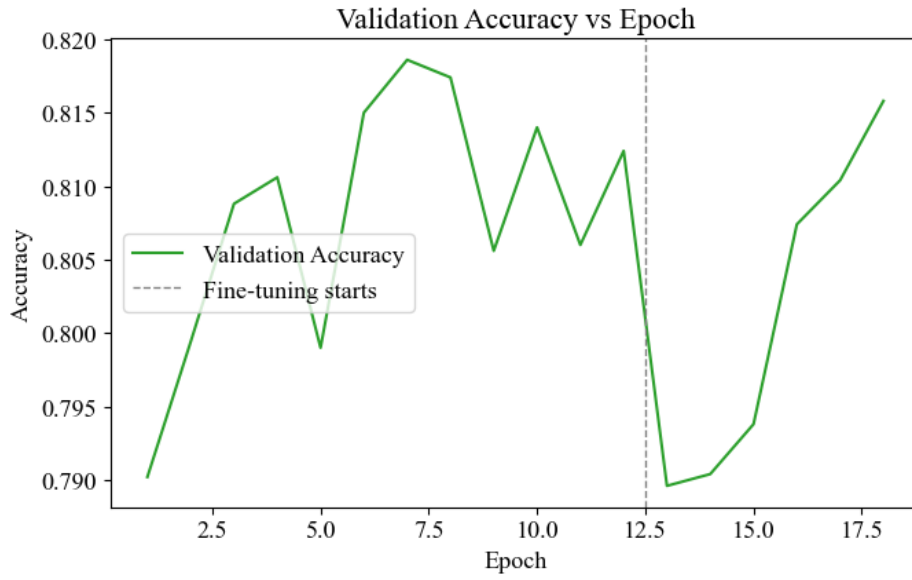


Figure 5: Validation Accuracy vs Epoch

Observation: Validation accuracy follows a noisier version of the same pattern, peaking near 81.86% during the frozen stage, dropping to about 78.9% right after fine-tuning starts, and recovering to 81.58% by the last epoch – close to, but not above, the earlier peak.

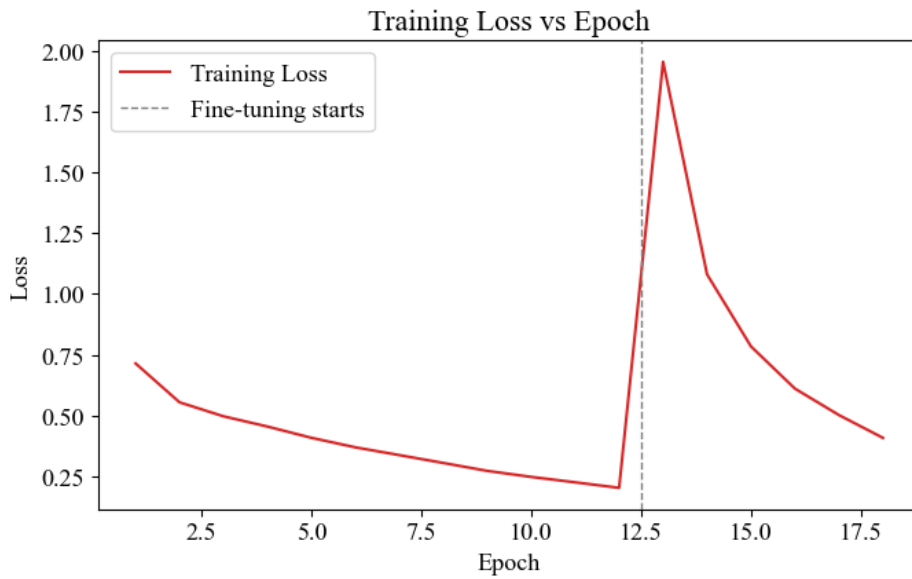


Figure 6: Training Loss vs Epoch

Observation: Training loss decreased steadily from about 0.72 to 0.20 during frozen training, jumped back up to around 1.95 at the start of fine-tuning, and decreased again to 0.41 by the last epoch, mirroring the accuracy curve.

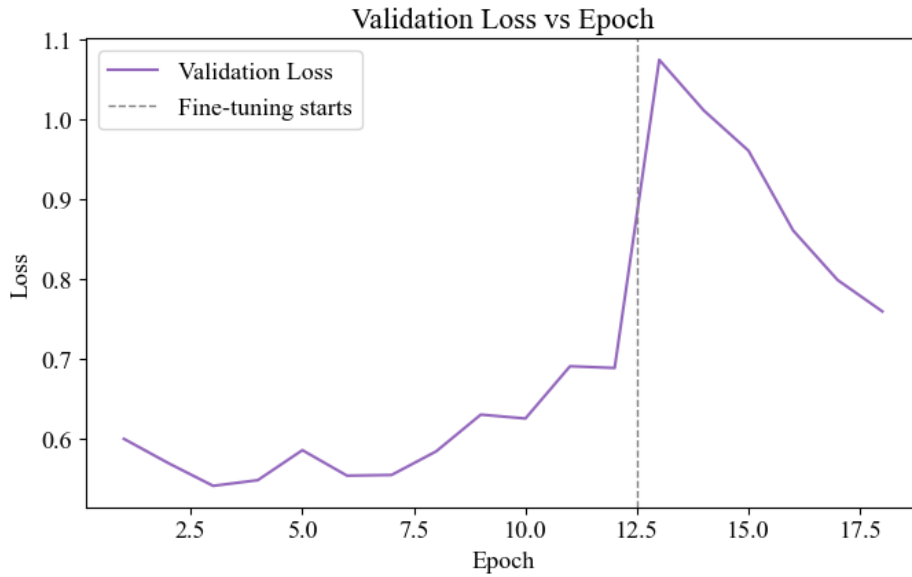


Figure 7: Validation Loss vs Epoch

Observation: Validation loss decreased from about 0.60 to 0.54 during frozen training, rose sharply to around 1.08 once fine-tuning started, and settled at 0.76 by the last epoch, staying above the lowest point reached earlier. Together, these four curves show that unfreezing the last block and lowering the learning rate caused a real but temporary disruption, and that six fine-tuning epochs were not quite enough to recover past where the frozen model already was, which lines up with the small drop in final test accuracy.

Task 5: Model Evaluation

The final fine-tuned model was evaluated on the full 10,000-image test set.

Metric	Value
Testing Accuracy	80.26%
Weighted Precision	80.28%
Weighted Recall	80.26%
Weighted F1-score	80.24%

Class	Precision	Recall	F1-score	Support
Airplane	0.81	0.84	0.83	1000
Automobile	0.90	0.91	0.90	1000
Bird	0.77	0.74	0.76	1000
Cat	0.64	0.65	0.64	1000
Deer	0.78	0.76	0.77	1000
Dog	0.71	0.70	0.70	1000
Frog	0.79	0.86	0.82	1000
Horse	0.86	0.81	0.83	1000
Ship	0.89	0.86	0.87	1000
Truck	0.88	0.90	0.89	1000
Accuracy	0.80			10000
Macro Avg	0.80	0.80	0.80	10000
Weighted Avg	0.80	0.80	0.80	10000

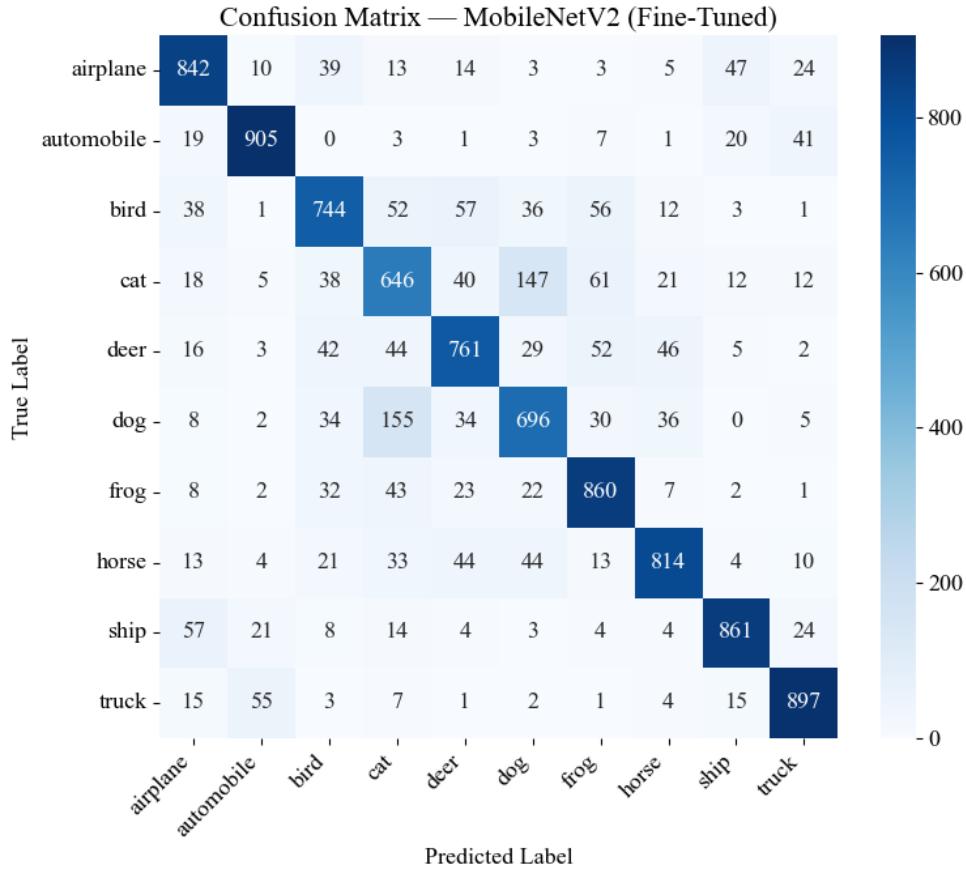


Figure 8: Confusion Matrix – MobileNetV2 (Fine-Tuned)

Observation: The largest single confusion in the matrix is between cat and dog: 147 cats were predicted as dog and 155 dogs were predicted as cat, which lines up with these two classes having the lowest recall in the table above (65% and 70%). Airplane and ship, and automobile and truck, also show some mutual confusion, which makes sense since both pairs share a similar overall shape and background.



Figure 9: Sample Misclassified Test Images

Observation: The misclassified examples shown here mostly belong to the same confusable pairs identified in the confusion matrix, and in several cases the true and predicted classes do look genuinely similar at this resolution.

16. Hyperparameter Study

Searching the full grid given in the manual (learning rate, batch size, epochs, optimizer, dense units and frozen mode) exhaustively would mean $2 \times 3 \times 2 \times 2 \times 2 \times 2 = 96$ combinations, which is too many to train one by one in a lab setting. Instead, 8 combinations were sampled at random and each was trained on a smaller stratified subset (400 images per class for training, 100 per class for testing) so the whole study would finish in a reasonable time. The results, ranked by validation accuracy, are given below.

LR	Batch	Epochs	Optimizer	Dense	Frozen	Val. Acc.
0.001	32	20	Adam	256	All	77.7%
0.0001	16	20	Adam	256	Partial	74.3%
0.0001	16	20	SGD	256	All	73.5%
0.001	16	20	Adam	128	Partial	69.4%
0.0001	32	10	SGD	128	All	66.2%
0.0001	64	10	SGD	128	All	61.7%
0.001	64	10	Adam	128	Partial	57.5%
0.001	16	10	Adam	128	Partial	52.8%

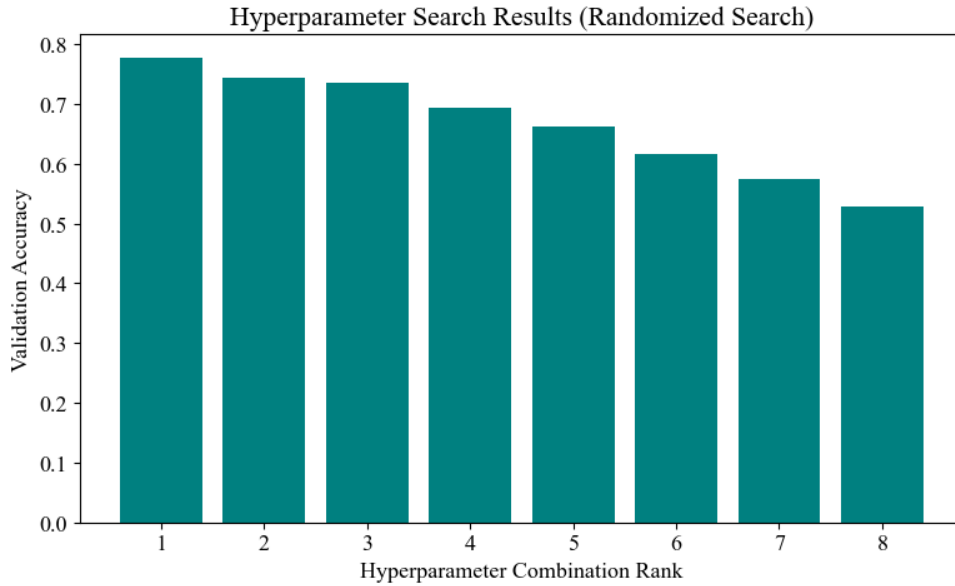


Figure 10: Hyperparameter Search Results (Randomized Search)

The best combination found was a learning rate of 0.001, batch size 32, 20 epochs, the Adam optimizer, 256 dense units, and the backbone fully frozen, reaching 77.7% validation accuracy.

Observation: The number of epochs shows the clearest pattern in this table – all four combinations trained for 20 epochs landed in the top four results (69.4% to 77.7%), while all four combinations trained for only 10 epochs landed in the bottom four (52.8% to 66.2%), regardless of the other settings. A dense layer of 256 units also tended to do better than 128 units among the top results. Learning rate, batch size, optimizer and whether the backbone was fully or partially frozen did not show as clear a pattern here, which is discussed further as Exercise 3 in Section 20.

17. Mandatory Plots

All of the plots required for this experiment – sample CIFAR-10 images, training accuracy, validation accuracy, training loss, validation loss, the confusion matrix and a set of misclassified images – are presented above in Section 15, alongside the task they belong to, each with its own brief observation.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy	86.69%
Testing Accuracy	80.26%
Precision	80.28%
Recall	80.26%
F1-score	80.24%
Training Time	409.02 s
Total Parameters	2,423,242

18.2 Comparison of Architectures

This table reports what was actually measured in this experiment. MobileNetV2 was trained on the full dataset as the main model; VGG16, ResNet50, the AlexNet-style network and the LeNet-5-style network were each trained on the smaller subset used for the Additional Exercises (6,000 training images, 1,500 test images, 8 epochs). LeNet-5-style and AlexNet-style were trained from scratch at CIFAR-10's native 32×32 resolution, since no pretrained weights exist for either; the other three used 96×96 resized inputs with frozen, pretrained ImageNet weights.

Model	Parameters	Accuracy	Training Time (s)
MobileNetV2 (main, full dataset)	2,423,242	80.26%	409.02
VGG16 (transfer, subset)	14,781,642	63.07%	69.90
ResNet50 (transfer, subset)	23,851,274	30.53%	50.90
AlexNet-style (from scratch, subset)	6,978,890	44.33%	33.01
LeNet-5-style (from scratch, subset)	83,126	47.60%	11.10

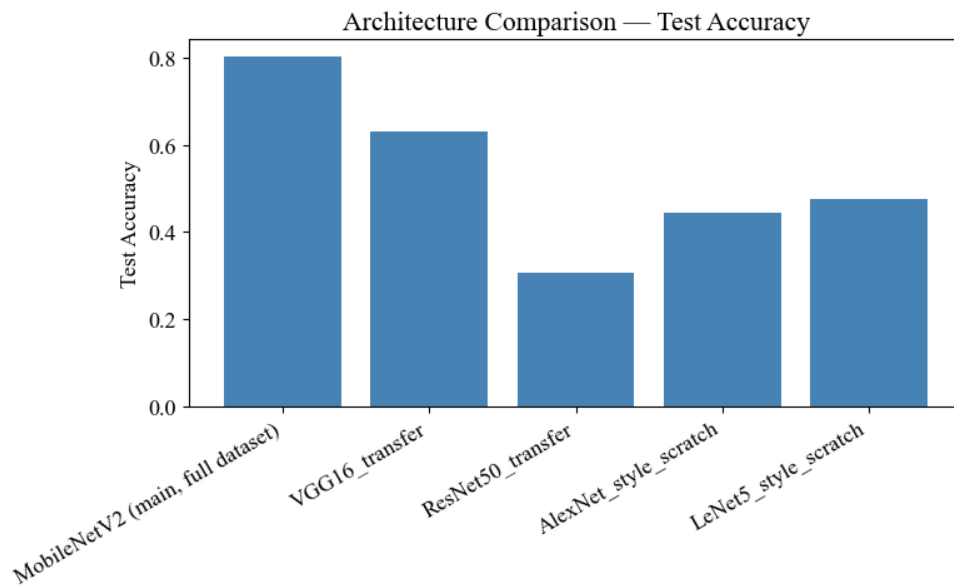


Figure 11: Architecture Comparison – Test Accuracy

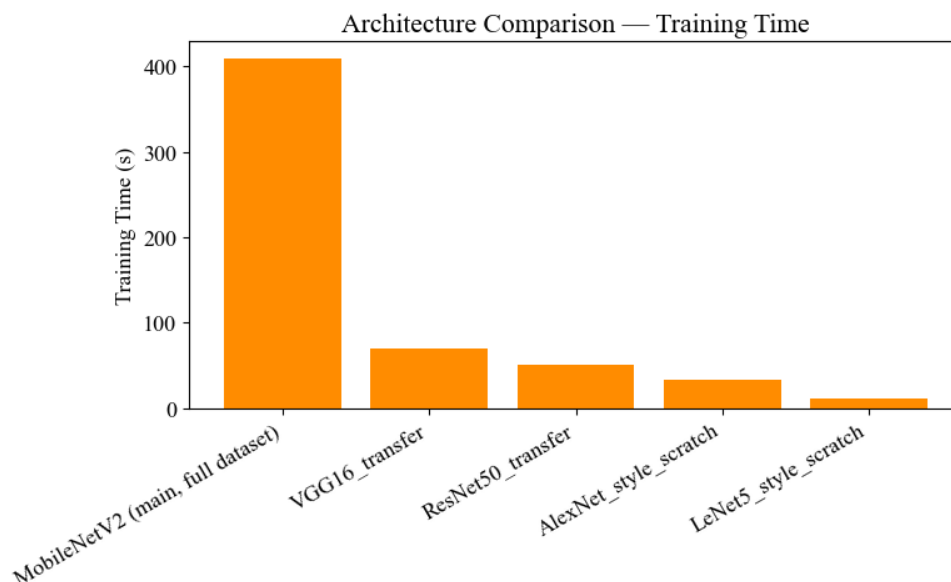


Figure 12: Architecture Comparison – Training Time

Observation: MobileNetV2 reached the highest accuracy by a clear margin, but it was also trained on far more data (45,000 images against 6,000) for more epochs (18 against 8), so this is not a fair comparison of the architectures on equal terms – it mainly shows what the full pipeline achieved. Among the four models trained on the same subset, VGG16 did noticeably better than ResNet50 (63.07% against 30.53%), even though both used a frozen pretrained backbone with the same size input and the same shallow head. One likely reason is that ResNet50 relies heavily on batch normalization layers throughout its convolutional blocks, and their statistics stay fixed once the backbone is frozen; VGG16 has no batch normalization in its convolutional layers, so it is less sensitive to being frozen in this way. It is also worth pointing out that the two from-scratch networks, LeNet-5-style and AlexNet-style, both reached higher accuracy than the frozen ResNet50 transfer-learning run on this same subset, despite having a fraction of its parameters. This suggests that, with only 6,000 training images and 8 epochs, training a small network directly on CIFAR-10 worked better here than training a shallow head on top of a frozen ResNet50 base.

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?** AlexNet was the first CNN to win the ImageNet competition by a large margin, and it showed that a deep network could be trained in practice by combining ReLU activations, which train faster than sigmoid or tanh, Dropout to control overfitting, and GPU implementation to make training large models feasible in a reasonable amount of time. It effectively proved that deep learning could outperform the hand-engineered computer-vision methods that came before it.
2. **Why does VGG16 use only 3×3 convolution filters?** Stacking several 3×3 filters gives the same effective receptive field as one larger filter (two 3×3 layers cover the same area as one 5×5 layer) while using fewer parameters and adding an extra non-linearity in between. This lets the network go deeper without the parameter count growing as quickly as it would with larger filters.
3. **Explain the advantages of the Inception module.** The Inception module applies several filter sizes (1×1 , 3×3 , 5×5) and pooling in parallel and concatenates the results, so the network can capture features at multiple scales in the same layer instead of having to commit to one filter size. The 1×1 convolutions used before the larger filters reduce the number of input channels first, which keeps the computational cost down even though multiple filter sizes are being used together.
4. **What is the purpose of residual learning?** Residual learning lets a layer learn the difference

between its input and its desired output, $F(x) = H(x) - x$, instead of learning the full mapping $H(x)$ directly. The skip connection then adds the input back, $F(x) + x$, which gives the gradient a direct path back through the network during backpropagation. This makes very deep networks much easier to optimize, since it avoids the vanishing gradient problem that made earlier deep networks hard to train.

5. **Differentiate LeNet and ResNet.** LeNet-5 is a shallow network with about 60 thousand parameters, built from two convolution layers and two average pooling layers, and designed for small greyscale digit images. ResNet50 is a much deeper network with around 25.6 million parameters, and its main innovation, residual skip connections, is exactly what makes that extra depth possible to train in the first place. In this experiment, the LeNet-5-style network trained here had 83,126 parameters, in the same range as the original, while the transfer-learning ResNet50 run kept its much larger frozen backbone.
6. **What is Transfer Learning?** Transfer learning means reusing a model that has already been trained on a large dataset, usually ImageNet, as the starting point for a new, usually smaller, task, instead of training a new network from scratch. The convolutional base is generally kept frozen so its learned features are reused directly, and only a new classification head is trained on the new dataset.
7. **Why is fine tuning required?** Features learned from ImageNet are general, but they are not perfectly suited to a different dataset such as CIFAR-10. Fine-tuning unfreezes some of the later layers of the base network, which tend to hold more task-specific features, and retrains them at a low learning rate so they can adjust slightly to the new dataset, on top of the new classification head that was already trained. In this experiment fine-tuning did not improve test accuracy (it went from 80.54% to 80.26%), which shows that fine-tuning is not guaranteed to help within a small number of epochs – if the learning rate or the number of unfrozen layers is not well suited to the new data, it can disturb the pretrained features before the model has time to settle back down.
8. **Explain the difference between dilated convolution and transpose convolution.** Dilated convolution keeps the output the same size as a normal convolution but spreads the kernel elements further apart, which increases the receptive field without increasing the number of parameters; it is mainly used to gather more context, for example in semantic segmentation. Transpose convolution does the opposite job: it increases the spatial size of the feature map, effectively learning how to upsample, and is mainly used to reconstruct or generate images, for example in autoencoders or generative networks.
9. **Why do pretrained models converge faster?** A pretrained model already has convolutional filters tuned to detect general features such as edges, textures and shapes, learned from a very large dataset. Starting from these weights instead of random initialisation means the network does not have to relearn these basic features from nothing, so fewer epochs are needed for it to reach a reasonable accuracy on the new task.
10. **Compare the computational complexity of LeNet and ResNet.** LeNet-5 is computationally light, with roughly 60 thousand parameters and only two convolution layers, so it trains quickly even on a CPU. ResNet50 is far heavier, with around 25.6 million parameters across 50 layers, and normally needs a GPU to train in a reasonable time. This difference shows up directly in this experiment: the LeNet-5-style network trained in 11.10 seconds on the subset, compared with 50.90 seconds for the ResNet50 transfer-learning run on the same subset.

20. Additional Exercises

Exercise 1: Transfer Learning Using VGG16

VGG16 was set up the same way as the main MobileNetV2 model – pretrained ImageNet weights, frozen convolutional base, Global Average Pooling followed by a Dense head – and trained on the smaller subset (6,000 training images, 1,500 test images) for 8 epochs. It reached **63.07%** test accuracy, with 14,781,642 total parameters, and took 69.90 seconds to train.

Exercise 2: Transfer Learning Using ResNet50

ResNet50 was set up and trained the same way, on the same subset. It reached **30.53%** test accuracy, with 23,851,274 total parameters, and took 50.90 seconds to train – noticeably lower accuracy than VGG16 under the same conditions, discussed further in Section 18.2.

Exercise 3: Adam vs SGD

This comparison is drawn from the Section 16 hyperparameter search, which already varies the optimizer across its sampled combinations.

Optimizer	Mean Val. Accuracy	Best Val. Accuracy	Combinations Sampled
Adam	66.34%	77.7%	5
SGD	67.13%	73.5%	3

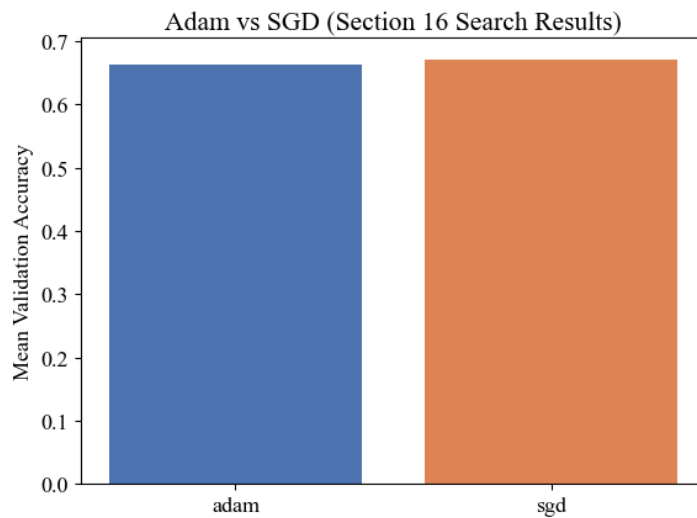


Figure 13: Adam vs SGD (Section 16 Search Results)

Observation: Across the 8 randomly sampled combinations, Adam and SGD ended up with a similar mean validation accuracy (66.34% against 67.13%), and the single best result overall came from Adam (77.7%). Since only 8 combinations were sampled in total and the other hyperparameters were not held fixed between them, this is not a controlled comparison of the two optimizers on their own – it mainly shows that neither optimizer was clearly better across the settings that happened to be sampled.

Exercise 4: Frozen Layers vs Fine-Tuning

This reuses the Task 3 and Task 4 results for the main MobileNetV2 model.

Configuration	Test Accuracy
Frozen backbone only (Task 3)	80.54%
Fine-tuned, last block unfrozen (Task 4)	80.26%

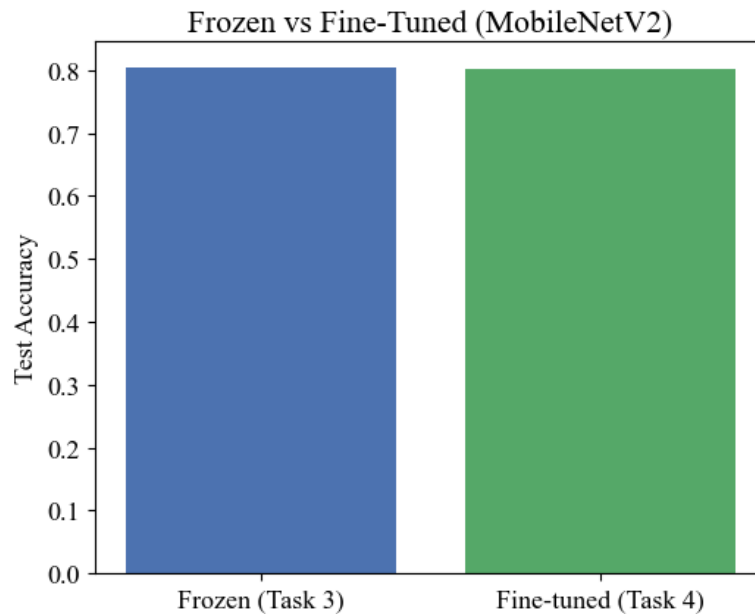


Figure 14: Frozen vs Fine-Tuned (MobileNetV2)

Observation: Fine-tuning the last block did not improve test accuracy here; it went down slightly, by 0.28 percentage points. As shown by the training curves in Section 15, unfreezing those layers caused training and validation accuracy to drop sharply before recovering, and six fine-tuning epochs were not enough for the model to fully return to, let alone exceed, where it already was with the backbone frozen.

Exercise 5: Precision, Recall and F1-score

This reuses the Task 5 evaluation of the final fine-tuned MobileNetV2 model: weighted precision of 80.28%, weighted recall of 80.26%, and weighted F1-score of 80.24%. The full per-class breakdown is given in the classification report table in Section 15.

Exercise 6: LeNet-5 vs AlexNet-Style vs ResNet50

Since neither LeNet-5 nor AlexNet has pretrained weights available in standard deep learning libraries, and neither’s original design matches CIFAR-10’s resolution (AlexNet expects 227×227 images, and LeNet-5 was designed for single-channel 32×32 digit images), both were implemented here as their usual CIFAR-10 adaptations and trained from scratch: LeNet-5-style kept the original two-convolution, two-average-pool, three-fully-connected structure, and AlexNet-style kept the five-convolution-block structure but with smaller kernels sized for 32×32 inputs. Both were trained on the same subset as ResNet50 (6,000 training images, 1,500 test images, 8 epochs).

Model	Parameters	Accuracy	Training Time (s)
LeNet-5-style (from scratch)	83,126	47.60%	11.10
AlexNet-style (from scratch)	6,978,890	44.33%	33.01
ResNet50 (transfer learning)	23,851,274	30.53%	50.90

Observation: Both from-scratch networks reached higher accuracy than the frozen ResNet50 transfer-learning run on this subset, and LeNet-5-style did so with a fraction of a percent of ResNet50’s parameter count. This lines up with the observation already made in Section 18.2: with only 6,000 training images and 8 epochs, a small network trained directly on the data ended up more effective here than a much larger pretrained network with only its head trainable.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>
9. Alex Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (CIFAR-10/CIFAR-100 dataset description): <https://www.cs.toronto.edu/~kriz/cifar.html>